

PROXMOX AUTOMATION SERIES · PHASE
4

The Request Form

One typed form on Semaphore UI: profile, hostname, IP, VLAN, and a custom disk list. Behind it, Ansible-native provisioning replaces Terraform and a delivery gate signs the handover. Twenty-plus field runs deep, every layer carries its receipt.

PVE 9.2 · SEMAPHORE UI 2.19.11 · ANSIBLE-CORE 2.21
COMMUNITY.PROXMOX 2.0.0 · SQLITE DIALECT · SURVEY FORM
AUGUST 2026 · COMPANION REPO: PROXMOX-AUTOMATION

FILL · RUN · DELIVER

Contents

1. What Phase 4 Adds	1
2. The Engine: Ansible-Native Provisioning	4
3. sem-01: the Semaphore Container	10
4. The Git Origin on ctrl-01	12
5. Wiring the UI: Objects and the Form	15
6. First Deploy, Two Ways	18
7. The Delivery Gate: 07_delivery_check.yml	23
8. Troubleshooting	25
9. Where the Project Stands	30

1. What Phase 4 Adds

This is the fifth module of the Proxmox Automation Series, and the one the whole series was pointed at: the request form. Phases 1 through 3 built the pieces, one closed phase at a time, and each ended with its claims field-verified. A template exists on NFS. Clones come up with cloud-init networking. Serial-tagged disks converge into mounted XFS. Agents install, verify themselves, and forward logs in the QRadar pattern. What remained was the part the engineers actually touch twenty to ninety times a day: the intake. A request arrives, someone types specifications into a form, and a fully configured VM comes out the other end with a receipt.

Edition 2 of this guide carries two rounds, and the difference between them is the difference between a plan and a pipeline. Round one, 28 August 2026, shipped the plan: the playbooks, the container, the wiring, and a guide written the series way, claims verified in a sandbox before they were printed. Round two, 29 August, was the field campaign: twenty-plus Semaphore runs against the real cluster, thirteen numbered field findings, a token rotation, and at 18:54 local time the first fully green run, form to delivery-checked VM with the receipt waiting at the log receiver. This edition keeps round one's chapters where they were and folds round two's evidence into them, the same treatment the Phase 3 guide received. Errors get kept; that is the house rule.

The original Phase 4 plan, as recorded in the series notes, was a custom web application: an Angular frontend, a FastAPI backend, a job engine, a live timeline. That plan was abandoned during planning week, deliberately, and the reasons are worth more than the plan was. Two findings did the abandoning. The first came from the ready-made UI evaluation: the wheel already existed, and the leading candidate was healthier than expected. The second came from the Ansible collection research, and it quietly retired Terraform from the provisioning path. Both findings were verified against primary sources on 28 August 2026, not read off a blog.

Finding one: AWX is frozen, Semaphore is not, and the form exists

The ready-made-UI shortlist had two names on it. AWX, the Red Hat sponsored upstream of the commercial Ansible Automation Platform, is the famous one: 15.5 thousand stars, a real web UI, workflow builder, RBAC, surveys. Semaphore UI is the smaller one: a single Go binary, a Vue frontend, task templates, environment variables, and, as of this year, typed survey variables. The GitHub release API told the story that decides it:

Question	AWX (ansible/awx)	Semaphore (semaphoreui/semaphore)
Last stable release	24.6.1, published 2024-07-02; no release since	v2.19.11, published 2026-08-27; releases every 1-2 weeks
Operator status	"releases paused during a large scale refactoring" (project statement, Sep 2025); Red Hat contribution withdrawn	Active daily development, 14k stars, MIT license
Install footprint	Kubernetes (k3s) plus operator; roughly a 4 GB VM for the lab	One Go binary plus systemd; fits the existing LXC pattern
Request form	Surveys: strings and a few types; no dynamic disk list either	Survey variables: string, integer, enum, multiline text, secret
Audit trail	Strong (job history, credentials, RBAC)	Strong for this use (every task stores who, when, which form values, full output)

Table 1. The evaluation that redirected Phase 4, verified from the GitHub API and each project's release notes on 28 Aug 2026.

The honest summary: AWX would have worked technically. Every requirement maps to an AWX feature, mostly without workarounds. But binding a twenty-to-ninety-deploys-a-day operations pipeline to a release train that has not moved in two years is the kind of decision that looks free today and costs a weekend next year. Semaphore is not a compromise here; its survey feature covers every field the request form needs (chapter 5), and its release cadence is the opposite of frozen. The custom-UI plan, meanwhile, failed on ownership arithmetic: a bespoke web app is another system to build, test, and live with, and it duplicates what Semaphore already ships.

0

a custom web UI at all. The decision inverted the series' usual build-donot-buy reflex because the thing being built was not differentiating: the value lives in the playbooks, and the playbooks run under any runner.

Finding two: the Proxmox collection grew up, and Terraform retired

The second finding came from checking whether the provisioning path could drop its second language. In the community.general era, the answer was no: the proxmox modules there could clone, but disk management was too thin for the Phase 2 serial design. The proxmox content has since split into its own collection, community.proxmox, and its 2.0.0 line (verified from the Galaxy API and the module source itself) closes the gap. The proxmox_disk module takes a serial parameter natively, up to 20 bytes, which is exactly the PVE limit; proxmox_kvm clones templates and applies full specifications; proxmox_vm_info answers existence queries. The entire Phase 1-to-3 Terraform output can now be

produced by Ansible alone.

That retires Terraform from the provisioning path, and the retirement is about state, not fashion. The lab's VMs are created by the pipeline and deleted by humans, outside it. Every terraform plan after such a deletion shows drift, and every import-or-taint dance is operator time spent reconciling a state file with reality. The Ansible-native path has no state to reconcile: a VM either exists (looked up by name) or it does not, and the playbook handles both cases explicitly. The terraform/ directory stays in the repository, runnable for reference and for the template build, but the deploy flow no longer touches it.

Phase 4 object	Replaces	Notes
00_provision_vm.yml (Ansible-native)	terraform apply of vm.tf	clone, specs, cloud-init, serial disks, start; no state file
Semaphore on sem-01 (CT 901)	the planned Angular + FastAPI app	typed survey form, task audit, zero custom code to own
07_delivery_check.yml	the manual handover checklist	mounts, services, forwarding, tagged message; green output is the record
profiles.yml (db-standard, web-small, app-medium, custom)	per-request tfvars editing	the preset layer of the form; serials derived from roles
semaphore/ (wiring reference + env template)	nothing; new	the UI objects, the survey design, the security notes
tasks/wait_for_apt.yml (pipeline era, 29 Aug)	nothing; new	the lock-based first-boot apt gate included by 01/02/04/05 before any package task; findings #14 and #16, chapter 2
vars/agent-layer.yml (pipeline era, 29 Aug)	group_vars loading on the deploy path	the agent knobs carried by plays 04/05/06/07 via vars_files so both run modes read one file; finding #18, chapter 6
field-hardened 00 (guards, pins, timeouts)	the blog-post version of the same play	timeout 300, the ide2 guard, local-lvm cloud-init, the vCPU-cap assert, the adoption MAC pin; findings #9 to #13 and #17, chapter 2

Table 2. What shipped, what each piece replaces, and the three rows the field campaign added on 29 August.

What was verified before this document was written

The series rule is that claims get verified, and Phase 4 had an unusual amount of verifying available because the artifacts themselves could run in a sandbox. The Semaphore v2.19.11 binary was downloaded, configured, migrated, given an admin user, and started; it answered HTTP 200. The one

path that validation never exercised was the login itself, and the field collected that debt within minutes of the first sign-in: the request answered HTTP 500 until the config carried the session cookie keys (chapter 3 and chapter 8 record the gap and the fix; the sandbox now tests the login endpoint too). The community.proxmox 2.0.0 tarball was pulled from Galaxy and its module source read line by line for the behaviors the playbook depends on. Every playbook passed syntax checking under ansible-core 2.21, and the profile resolution logic ran as four standalone logic tests (custom, db-standard, web-small, app-medium), which caught two real bugs before they could reach the lab. Both bugs are documented in chapter 8 with their fixes.

Then the field wrote its own section. The first evening, 28 August, took the pipeline from six failed runs to the first VM that booted, answered SSH, and handed play 01 a guest to configure; every failure on the way became a numbered finding and then a guard, a timeout, or a comment in the shipped files. The second day, 29 August, took it the rest of the way: the first-boot apt race, the adoption-path SSH death, the inventory Semaphores never loads, one invalid module parameter, and at 18:54 the first fully green run, whose receipts (the play recap and the delivery tag at the receiver) are printed in chapters 6 and 7 exactly as the field filed them. Thirteen findings, all closed; the ledger is chapter 6's Table 11.

2. The Engine: Ansible-Native Provisioning

The heart of Phase 4 is one new playbook, 00_provision_vm.yml, and the module behaviors it is built around. This chapter walks the playbook step by step, records the three community.proxmox behaviors that shaped it, and then documents the layer the field campaign added on top: the guards, the pins, and the first-boot apt gate that turned a demonstrably racy first deploy into a boring one. Those behaviors are not documented prominently anywhere; they were found by reading the collection source, and the guards were found the honest way, by being needed.

The steps, as shipped

The play runs on localhost with connection: local; it speaks to Proxmox over HTTPS, never SSH. The steps below are the shipped play's own numbering, comments included, and two of them (2b and the MAC pin inside step 4) did not exist in the first edition of this guide; the field added them.

- Validate the request. Profile name, DNS-safe hostname, dotted IPv4, VLAN range; for custom profiles also the vCPU and RAM enum sets and the disk JSON, plus the node's vCPU cap (vm_max_vcpu) so a doomed request dies before the clone spends minutes. The form is UX; this assert is the gate.
- Resolve the profile into a specification: vCPU, RAM, and the disk list with serial, VG, LV, and mount derived from each role.

- Look up existing VMs with that name (`proxmox_vm_info`, `config: current`). Found means reconfigure; not found means clone. This lookup is the idempotency guard, and the same call now extracts two facts the re-run path needs: whether the VM already carries its cloud-init drive (#10) and the net0 MAC it booted with (#17).
- Admission control (Phase 5a): IP collision against every VM's `ipconfig0`, a TCP/22 liveness probe against the requested address, and the fleet's allocated vCPU/RAM budget. A refusal prints its math and dies before the clone, leaving nothing behind.
- Clone the golden template (full copy on `nfs-shared`), then apply the specification in a second task: CPU, RAM, net0 with optional VLAN tag and, on the adoption path, the pinned existing MAC; the cloud-init drive on `local-lvm` (#11); `ipconfig0`, SSH keys, DNS, agent, onboot, tags.
- Attach the serial-tagged data disks on `scsi1..n` (`proxmox_disk`), one per role, with `create: regular` so re-runs are no-ops on existing disks.
- Start the VM (timeout 300, finding #9), wait for SSH, and add `host` it into the provisioned group so plays 01 through 07 pick it up in the same run.

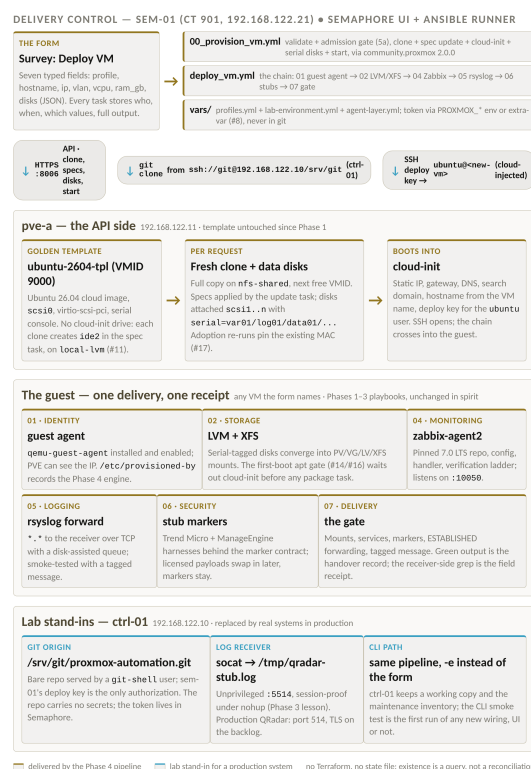


Figure 1. The Phase 4 delivery path: one form, one pipeline, one receipt. sem-01 runs Semaphore and the playbooks; ctrl-01 hosts the git origin and the log receiver stub; pve-a serves the API.

Three module behaviors that shaped the playbook

The first behavior is the one that would have been invisible until the first production run. In `proxmox_kvm`'s clone branch, the module forwards only six parameters to the clone API call: format, full, pool, snapname, storage, and target. Everything else passed alongside clone, the cores, the memory, the network, the cloud-init settings, is silently dropped. A single-task "clone with specs" that looks correct in every blog post produces a clone with the template's defaults. The playbook therefore clones first, then applies the specification in a second task with `update: true` against the new VMID.

The second behavior lives in that update task. The module's update path deliberately skips disks and network interfaces unless `update_unsafe: true` is set; the safety rail exists because rewriting `net0` on a production VM is risky. A fresh clone has nothing to clobber, and the specification explicitly includes `net0` and `ide2`, so the playbook sets the flag. On re-runs against an existing VM the values are identical, and the configuration call is a no-op, with one field-taught exception: the `net0` line must carry the VM's existing MAC or PVE treats the omission as an instruction to regenerate it (finding #17, chapter 6).

The third behavior is idempotency, or the lack of it. In clone mode the module does not check whether a VM with the target name already exists; it allocates the next free VMID and clones again. A re-run of a naive playbook therefore produces a second VM with the same name and a different ID. The playbook level guard in step 3 is not an optimization; it is what makes the whole chain re-runnable. The explicit-VMID variant has its own semantics: passing a `newid` that already exists exits with `changed: false` and that VMID, which the playbook treats as an existing VM and reconfigures.

Behavior	Source evidence	How the playbook answers
Clone drops spec parameters	<code>create_vm()</code> clone branch: only format, full, pool, snapname, storage, target reach the API	Two tasks: clone, then <code>update: true</code> with the full spec
Update skips net and ide	update path strips net, scsi, virtio, ide and more unless <code>update_unsafe: true</code>	<code>update_unsafe: true</code> on a fresh clone; identical values on re-runs
Clone mode is not idempotent by name	<code>newid</code> defaults to next-free; no name check in the clone path	<code>proxmox_vm_info</code> by name first; found means reconfigure, not clone
Cloud-init drive must exist before its parameters	the Phase 1 template never had one (initialization lived on the workload)	<code>ide2: local-lvm:cloudinit</code> created in the same update task, and only when the VM does not carry one yet (#10, #11)

Table 3. Module behaviors from the `community.proxmox 2.0.0` source, and the playbook's answer to each.

Profiles: the preset layer

The request form does not ask for disk serials, VG names, or mount points, and it never will. It asks for a profile, and the profile resolves to a disk list whose entries carry only a role and a size. Everything else is derived: role data becomes serial `data01`, volume group `vg_data`, logical volume `lv_data`, mount point

/srv/data. The derivation is deliberately identical to the field-proven Phase 2 layout, so the db-standard profile produces exactly the disk set the lab already ran with, and the unchanged 02_storage.yml builds either path's disks.

Profile	vCPU / RAM	Disks (role -> mount)
db-standard	8 / 32 GB	var -> /srv/var, log -> /srv/log, data -> /srv/data, backup -> /srv/backup
web-small	2 / 4 GB	none (the storage play ends itself cleanly)
app-medium	4 / 16 GB	app -> /srv/app
custom	from the form enums	from the form's JSON; roles limited to var, log, data, backup, app, scratch

Table 4. The profile set. Sizes are production-shaped; lab validation should use custom with small sizes.

One field caveat travels with the table: db-standard's 8 vCPU is a specification the lab's node policy refuses at start time (finding #12). The assert on `vm_max_vcpu` exists precisely so that refusal happens in seconds at the top of the run, with a message that names the cap, instead of minutes later at `qm` start with one that does not. In the field the cap was raised from 2 to 4 vCPU and the lab's proving runs used the custom profile at 2 vCPU / 4 GB with a single 10 GB data disk, which is also the shape the CLI example in chapter 6 uses.

The chain: `deploy_vm.yml`

Semaphore runs exactly one playbook per task template, so the chain lives in `deploy_vm.yml`: seven `import_playbook` lines in fixed order, 00 through 07. One deliberate omission: `03_storage_verify.yml` is not in the chain, because it reboots the VM to prove boot persistence, and a reboot in the middle of every deploy would double delivery time for no daily value. It remains available as its own template for on-demand proof, exactly as in Phase 2. The chain also documents the two-inventory split: the deploy flow uses `provisioning.yml`, whose provisioned group is empty on purpose and fills at runtime via `add_host`; the maintenance inventory `hosts.yml` keeps the known VMs, so a deploy can never accidentally touch them. A root-level `ansible.cfg` mirrors the ansible one and exists for how Semaphore invokes the runner: it clones the repository and executes from the clone root.

0

a second orchestration language. Everything Terraform did for this pipeline is now done by the same tool that configures the guest, against the same API, with no state file to reconcile.

The field-hardening layer (findings #9 to #13)

The first edition of this chapter ended with the module behaviors, and the field replied with five more lessons, one per failed start, on the evening of 28 August. None of them changed the design; all of them changed the playbook. The pattern is the same every time: the failure mode was invisible in the sandbox because the sandbox never crosses the boundary where it lives, and the fix is small, explicit, and commented where it stands.

Finding	The failure as the field saw it	The shipped answer
#9 start timeout	the start task died at the module default of 30 s while ide2's cloud-init ISO was still being generated	timeout: 300 on the start task
#10 ide2 re-send	a re-run answered HTTP 500 from PVE: re-applying ide2 re-ALLOCATES the cloudinit disk that already exists	vm_has_cloudinit read from config: current; ide2 sent only when the VM does not carry one yet
#11 cloud-init on NFS	qm start hung deterministically at "generating cloud-init ISO" when the drive lived on nfs-shared	proxmox_cloudinit_storage: local-lvm; the ISO is small and lock-sensitive, local storage is its right home
#12 node vCPU cap	qm set accepted the spec, qm start refused it: the node policy caps per-VM vCPU (MAX 2, later raised to 4)	vm_max_vcpu in lab-environment.yml, asserted before the clone so the refusal costs seconds, not minutes
#13 hot-applied spec	a memory shrink applied to the running VM wedged it; a running VM is not a reconfigure-everything target	clean-slate destroy plus re-provision for spec changes; re-runs re-apply identical values only

Table 5. The 28 August start failures and the guards they became. Every fix is a comment in the shipped 00_provision_vm.yml.

Two of these deserve their paragraph. Finding #10 is a peculiarly PVE-shaped trap: the API call that creates the cloud-init drive is not the same as the one that updates it, and sending the create form to an existing drive answers 500 with an error that reads like a bug and is actually a policy. The guard reads the VM's current config in the same lookup the idempotency guard already used, so the fix cost one fact and one conditional. Finding #11 is the one that looked like bad luck twice before it looked like a cause: an ISO regeneration is a small write, but on NFS it is a small write with a lock in it, and qm start holds that lock's patience. Moving the cloud-init drive to local-lvm turned the hang into a boot; the comment in lab-environment.yml records the reasoning so nobody moves it back for tidiness.

The first-boot apt gate (findings #14 and #16)

The hardest bug of the campaign was not in the playbook at all; it was in the gap between two apt users. A fresh Ubuntu 26.04 clone spends its first minutes running cloud-init's own package pass, and Ubuntu runs unattended-upgrades on top. The pipeline's apt tasks, arriving minutes later, met a held dpkg lock and

died with rc=100 and a 60-second lock timeout from inside the module. That was finding #14, and its first fix, a loop that waited for apt-shaped process names to leave the table, was reasonable and wrong.

Wrong because unattended-upgr on this distribution is not a transient worker; it is the permanent unattended-upgrades daemon, and it never leaves. Run #30 proved it exhaustively: sixty polls over five minutes, every single one reporting nothing but the idle daemon, zero apt or dpkg workers, in a window where the upgrade timers provably had not fired. A name-based gate cannot clear on a box where one of the names always matches. The fix that shipped, gate v3, stops reading process names and reads what actually blocks an apt call: the lock files.

RUN #30, 29 AUGUST 2026, 13:38 TO 13:43 (CONDENSED TO THE FAILING TASK)

```
TASK [Wait for any lingering apt/dpkg activity to clear] ****
FAILED - RETRYING: Wait for any lingering apt/dpkg activity \
to clear (60 retries left).
... every poll for five minutes reports the same thing ...
FAILED - RETRYING: Wait for any lingering apt/dpkg activity \
to clear (1 retries left).
fatal: [vm-faz4-01]: FAILED! => {"attempts": 60,
"stdout": ["unattended-upgr"], ...}

# sixty samples, five minutes, zero apt/dpkg workers: the
# name-based gate was waiting for a process that never leaves.
```

Gate v3, now tasks/wait_for_apt.yml, has two layers and a second gatekeeper. First it waits on the orchestrator: timeout 1800 cloud-init status --wait, because cloud-init's apt pass runs inside cloud-init, so its exit is the signal that the pass is done; the rc is reported, not enforced (rc=2 means finished with non-fatal warnings, which is informational by design). Then the lock layer: fuser on the four dpkg/apt lock files, systemctl is-active on the apt-daily oneshots, and a name fallback that only fires on boxes without fuser and deliberately excludes the daemon. Busy polls now print the reason, lock holder pid or active unit, so the next person to watch a first boot watches something legible. The window is 90 polls at 10 s. The second gatekeeper is lock_timeout: 1800 on the four real apt call sites, so even a wave the gate waves through cannot kill the install that follows it.

THE V3 GATE'S FIRST TWO FIELD PROOFS, 29 AUGUST (KEY LINES)

```
# run #31, 17:06, a TRUE first boot (fresh clone):
TASK [Wait for cloud-init to finish (its apt pass runs
inside it)] *****
ok: [vm-faz4-01]
TASK [Wait for any lingering apt/dpkg activity to clear
(lock-based)] *****
ok: [vm-faz4-01]

# run #36, 18:35, adoption re-run, same shape: cloud-init
# done instantly, the gate IDLE on its first poll, no retry
# lines at all. The permanent daemon stopped counting.
```

The gate is included by plays 01, 02, 04 and 05 ahead of their first package task, so every apt in the chain stands behind it. Its story also produced the campaign's smallest, most instructive bug: the same patch that added `lock_timeout` to the apt tasks also added it to `apt_repository`, which does not accept the parameter, and the error waited three runs to surface because play 04 had never reached that task inside the pipeline. That is finding #19, and it belongs to chapter 6, where the runs that exposed it are on the ledger.

3. sem-01: the Semaphore Container

Semaphore lives in its own unprivileged LXC container, CT 901, created by `scripts/create-sem-01.sh` on `pve-a` and configured by `scripts/sem-01-setup.sh` inside it. The sizing question has a boring answer: Semaphore is one Go binary with an embedded SQLite database, and the Ansible runs happen in the same container; 2 vCPU and 2 GB run the lab comfortably, and the script bumps nothing because nothing in the lab needs it.

The scripts encode the install decisions so the container can be rebuilt in minutes, and every non-obvious decision in them was verified against the artifacts rather than a blog. The `.deb` from the GitHub release was downloaded and inspected: it contains exactly one file, `/usr/bin/semaphore`, so the setup script creates the systemd unit itself, from the unit template in the official "Run as a service" documentation. The binary was then run for real: migrations applied, an admin user created with the `users add` subcommand, the server started and answered HTTP 200. Two findings from that session are now baked into the script and documented here because they will otherwise cost someone an evening.

The port must be a string

The first run of semaphore migrate against a hand-written `config.json` failed with a Go panic: `json: cannot unmarshal number into Go struct field ConfigType.port of type string`. The port field must be quoted, `"port": "3000"`, not `3000`. The panic is loud, the message names the field, and every example config in the wild seems to have the string form, but a JSON author writing it fresh from first principles will write a number. The setup script writes the config with the quotes; the error and its fix are in the troubleshooting matrix for whoever ignores the script.

The cookie keys must exist, or nobody logs in

The fourth field finding arrived at the login form, and it is the subtlest of the four: v2.19 signs its session cookies, and the binary requires two secrets in the config, `cookie_hash` and `cookie_encryption`. Without them the server starts, the form renders, the password verifies, and the sign-in still dies with HTTP 500 and "Failed to create session", because the cookie cannot be encoded. Two details make it worse than it sounds. The field names circulating in older guides, `cookie_hash_key` and `cookie_encryption_key`, belong to earlier versions and are silently ignored now; the working names above were read out of the binary

itself. And semaphore setup, the interactive wizard, generates both keys, so everyone who follows the official path never sees the failure, and everyone who writes a minimal config by hand meets it at the worst possible moment, one field before the dashboard. The setup script generates both keys with `od -N` (the pipefail-safe bounded read; `head -c` caused bug one) and writes them into the config; the sandbox validation now proves the login round trip, 204 plus a Set-Cookie header and an authenticated follow-up call, not just the HTTP 200 of the front page.

SQLite is the default, and that is fine

The current Semaphore configuration documentation lists the dialect values as `sqlite` (the default), `postgres`, and `mysql`. For a single-container lab the default is the right call: no database server to install, patch, or back up separately, and the database file lives in `/var/lib/semaphore` where a `pct` snapshot captures it. The switch to PostgreSQL is a set of environment variables, `SEMAPHORE_DB_DIALECT` and friends, so the upgrade path exists if remote runners or high availability ever arrive; the guide records that path and deliberately does not walk it.

SEM-01: /ETC/SEMAPHORE/CONFIG.JSON (WRITTEN BY SEM-01-SETUP.SH)

```
{
  "dialect": "sqlite",
  "sqlite": { "host": "/var/lib/semaphore/semaphore.db" },
  "port": "3000",
  "tmp_path": "/var/lib/semaphore/tmp",
  "demo": false,
  "cookie_hash": "<64 hex chars, generated at install>",
  "cookie_encryption": "<32 hex chars, generated at install>"
}
```

The service runs as a dedicated semaphore user, never root, with `WorkingDirectory` set to the data directory. The setup script generates the admin password when none is supplied, prints it once, and expects it changed on first login. It also generates the deploy keypair at `/home/semaphore/.ssh/id_ed25519`; that keypair is the pipeline's identity, one key doing two jobs in the lab: it authenticates sem-01 to the git origin on ctrl-01, and its public half is injected into every new VM by cloud-init so the playbooks can SSH in. The production-grade split, a separate key for each job, is a five minute change documented in `semaphore/README.md`.

0

a database server in the container. SQLite is the project's own default dialect; the file sits next to the job data in `/var/lib/semaphore`, and one `pct` snapshot captures the whole service state.

The Ansible environment inside the container

sem-01 doubles as the Ansible control node for the deploy flow, so it needs the toolchain. Ubuntu 26.04 blocks pip system-wide installs under PEP 668, the same externally-managed-python wall this series met in Phase 1, and the clean answer here is apt: ansible, python3-proxmoxer, python3-requests, python3-jmespath, and python3-netaddr all come from the distribution repository. If python3-proxmoxer is ever absent from the repository, the fallback is a venv under the service user; the setup script notes the path. The distribution's proxmoxer (2.2.0 at the time) also turned out to sit below the floor community. proxmox 2.0.0's newer info modules declare; a single pip install --break-system-packages proxmoxer==2.3.0, pure Python, no daemon, lifts it. The deploy path never noticed because proxmox_kvm and proxmox_vm_info do not declare the floor; the audit playbook's cluster modules did.

The collections taught the sharper lesson, and it is now a pinned line in requirements.yml. Ubuntu's ansible package BUNDLES an older community.proxmox (1.4.0) in the system path, and an unpinned bare-name install lets ansible-galaxy consider the requirement already satisfied: "Nothing to do. All requested collections are already installed", with 2.0.0 never arriving and proxmox_disk's serial support missing with it. The fix is the explicit version pin community.proxmox:>=2.0.0, and the install target matters just as much: /usr/share/ansible/collections, the HOME-independent system search path. The first installs went to the service user's home and worked on the command line, but the task runner's environment did not always resolve that home the same way, and a collection invisible to the task process is a collection that does not exist. The system path ends that entire class of ambiguity; the setup script installs the pinned set there, and the troubleshooting matrix carries the two failure shapes (chapter 8).

4. The Git Origin on ctrl-01

Semaphore needs a git repository URL; it clones the repository for every task run, and that is the whole point of this chapter: Semaphore runs on sem-01, and the only part of ctrl-01 it can reach is the network. The working copy that lived well enough as a plain directory through Phases 1-3 now needs a servable home: a bare repo behind a dedicated git user whose shell is git-shell.

CTRL-01 (ROOT): HOST THE REPOSITORY FOR SEMAPHORE

```

useradd -m -s /usr/bin/git-shell git
mkdir -p /srv/git
git init --bare --initial-branch=main /srv/git/proxmox-automation.git
# authorize sem-01's deploy key (sem-01-setup.sh step 7 output):
install -d -m 700 -o git -g git /home/git/.ssh
DEPLOY_KEY="ssh-ed25519 AAAAC3NzaC1lZDI1NTE5\
AAAAICaUb7zvNq8LGILOM96t9Le0BVjLjXuHnrydb+0k6fdU semaphore@sem-01"
echo "$DEPLOY_KEY" > /home/git/.ssh/authorized_keys
chown git:git /home/git/.ssh/authorized_keys
chmod 600 /home/git/.ssh/authorized_keys
# refresh the working copy from the CURRENT zip first (the drift
# trap is documented below), then publish:
cd /root/proxmox-automation
git init -b main && git add -A
git commit -m "phase 4: delivery pipeline"
git push /srv/git/proxmox-automation.git main
chown -R git:git /srv/git/proxmox-automation.git
git config --global --add safe.directory /srv/git/proxmox-automation.git

```

The chapter earned two errata rows in Table 8 the first time it ran for real, on 28 August 2026. Plain git init still creates a branch called master (the default changes only in Git 3.0) while the bare repo, the push command, and the Semaphore repository object all say main, so the original block pushed a branch that did not exist and git answered "error: src refspec main does not match any"; the corrected block above creates the branch with -b main from the start. The working copy itself had drifted too: the copy on ctrl-01 still held the Phase 3 tree, thirty files with no deploy_vm.yml and no provisioning.yml, and a push in that state would have handed Semaphore a repository without the playbook it exists to run. Refresh the working copy from the current zip before the first push, and read the commit list as a receipt: if deploy_vm.yml is not in what git add -A picked up, nothing downstream of the push can work.

The two lines after the push settle a boundary git has cared about since CVE-2022-24765: the repository was created by root, but the process that serves it over SSH runs as the git user, and git refuses to operate on a repository owned by somebody else. The failure mode is a clone that answers "fatal: detected dubious ownership". Handing the directory to the git user fixes the serving side; the safe.directory entry keeps root's local-path push working afterwards, because from that moment the repository is no longer root's either.

PVE-A (ROOT): PROVE THE CHAIN BEFORE THE UI

```

pct exec 901 -- bash -c "ssh-keyscan -H 192.168.122.10 \
>> /home/semaphore/.ssh/known_hosts"
pct exec 901 -- chown semaphore:semaphore \
/home/semaphore/.ssh/known_hosts
pct exec 901 -- chmod 600 /home/semaphore/.ssh/known_hosts
pct exec 901 -- sudo -u semaphore git ls-remote \
ssh://git@192.168.122.10/srv/git/proxmox-automation.git

```

The ls-remote at the end is the chapter's receipt: two lines, the same commit hash twice, for HEAD and for refs/heads/main. That one hash proves the entire chain: the deploy key, the authorized_keys file that holds it, git-shell, the bare repo, and its ownership. The keyscan above it exists because the semaphore user has never spoken to ctrl-01 before, and a non-interactive clone cannot answer an interactive host-key question. One prerequisite both commands assume: sshd listening on ctrl-01; ss -tln on ctrl-01 is the ten-second check, apt-get install -y openssh-server the fix if the container lacks it.

The campaign turned this receipt from a one-time ceremony into a habit, because the pipeline's fixes now travel the same road the first push did. Every field patch of 29 August followed the same discipline: apply the change on the ctrl-01 working copy, commit, push to the bare repo, then verify that rev-parse and ls-remote agree before firing the next run. The receipt below is the #17+#18 patch push, the one that carried the adoption MAC pin and the agent-layer vars file; the next Semaphore run greeted it with "Already up to date" and ran the new code.

THE PUSH RECEIPT, AS FILED ON 29 AUGUST 2026 (HASHES AS LOGGED)

```
root@ctrl-01:~/proxmox-automation# git rev-parse HEAD
a841f36007bfda089aa9984be26a00df8f575c23

root@pve-a:~# pct exec 901 -- sudo -u semaphore git ls-remote \
ssh://git@192.168.122.10/srv/git/proxmox-automation.git
a841f36007bfda089aa9984be26a00df8f575c23 HEAD
a841f36007bfda089aa9984be26a00df8f575c23 refs/heads/main
```

The git-shell shell is deliberate: the git user can serve repositories and nothing else, no interactive login, no command execution beyond the git protocol. In Semaphore the repository URL is ssh://git@192.168.122.10/srv/git/proxmox-automation.git with the deploy key selected as the access key. The push from the working copy is local, straight to the bare path, no SSH involved; only sem-01's clones go over the network. When the repository outgrows the lab, the same Semaphore object points at any hosted git URL and nothing else changes.

One habit worth keeping from day one: the working copy on ctrl-01 stays the editing surface, and pushes go to the origin before Semaphore runs them. The tokenless clone URL means the repository carries no secrets; the Proxmox API token lives in Semaphore's variable group, the deploy key lives in Semaphore's key store and the git user's authorized_keys, and the git history stays clean of credentials. The one token the lab created in Phase 1 was pasted into a chat window once, and the rotation that followed is chapter 5's story; the series owes the token no second exposure.

5. Wiring the UI: Objects and the Form

Everything in Semaphore hangs off one project, and the project needs five objects before the first task can run. Each is created once in the web UI; semaphore/README.md in the repository keeps the same reference for the day the UI needs rebuilding. None of the five contains a secret that git would mind holding, except the one that deliberately holds only secrets and lives in Semaphore, never in the repository.

Object	Name	Content
Key Store entry	deploy-key	the private half of sem-01's id_ed25519; used for repository access
Repository	proxmox-automation	ssh://git@192.168.122.10/srv/git/proxmox-automation.git, branch main, access key deploy-key
Inventory	provisioning	the content of ansible/inventory/provisioning.yml: localhost plus an empty provisioned group
Variable Group	proxmox-lab	the JSON of the four PROXMOX_* variables; the API token lives here and only here. v2.19 note below
Task Template	Deploy VM	Ansible type, playbook ansible/playbooks/deploy_vm.yml, the inventory and variable group above, and the survey

Table 6. The five Semaphore objects behind the form. The fourth row's delivery semantics are the subject of finding #8.

The variable group, and finding #8

The playbook reads the Proxmox connection through four names: PROXMOX_HOST, PROXMOX_USER, PROXMOX_TOKEN_ID, PROXMOX_TOKEN_SECRET. The community.proxmox modules document the first three as environment names themselves, which means the pipeline and the collection agreed on the convention before they met. On the CLI the same variables come from a shell export. In Semaphore, the first edition of this guide said, they come from the environment object attached to the template; the field replied, on the very first form-driven run, that v2.19 delivers a Variable Group's JSON values to the playbook as EXTRA-VARS, not as process environment variables. The playbook's token assert failed with an empty secret while the UI plainly held one, and the run died in seconds with the 401 the assert predicted.

The fix is one line per secret in vars/lab-environment.yml, and it is worth reading slowly because it is the pipeline's whole secret-handling philosophy in miniature: each value reads lookup(env, 'NAME') or NAME, environment variable first, extra-var of the same name second. A CLI run exports the variable and the first source answers; a Semaphore run delivers the Variable Group's JSON and the second source

answers; either way the token never appears in the repository, in a playbook, or in a task log, and a rotated token is a one-field edit in the UI. The lab's environment template, `semaphore/env/proxmox.env.example`, carries the same four names, and the Secrets tab is a third legitimate home for the token value if your Semaphore deployment prefers it.

The form: survey variables on the Deploy VM template

Field	Type	Values or example	Notes
<code>vm_profile</code>	enum	db-standard, web-small, app-medium, custom	the preset layer; most requests stop here
<code>vm_hostname</code>	string	vm-app-01	lowercase DNS-safe; doubles as the PVE VM name
<code>vm_ip</code>	string	192.168.122.60	dotted IPv4, no CIDR
<code>vm_vlan</code>	integer	0	0 means untagged
<code>vm_vcpu</code>	enum	2, 4, 8, 16, 32	custom profile only
<code>vm_ram_gb</code>	enum	4, 8, 16, 32, 64, 128	custom profile only
<code>vm_disks</code>	text (multiline)	JSON list, below	custom disks; overrides the profile list

Table 7. The seven survey fields. Enums where the pipeline has opinions; free text only where the request genuinely varies.

Two of these choices carry the whole design. The vCPU and RAM fields are enums, not number inputs, because the form should only ever offer what the pipeline allows: "the customer wants seven vCPUs" is a conversation, not a form value. And the disk list is the only free-form field, because custom storage requests genuinely vary; it accepts a JSON list of role and size pairs, and everything a mistyped disk spec would otherwise get wrong is derived instead of typed. The serials, the volume groups, the logical volumes, and the mount points are computed from the role names. Nobody can typo "data01" into a field that does not exist.

THE VM_DISKS JSON (CUSTOM REQUESTS ONLY)

```
[
  {"role": "data", "size_gb": 1024},
  {"role": "log", "size_gb": 50},
  {"role": "backup", "size_gb": 500}
]
```

The assert block in `00_provision_vm.yml` re-validates every field server-side, because the survey is only the UI path: extra-vars on the CLI and the task API both bypass it. The allowed roles are a closed list, sizes must be positive integers, roles must be unique, and the hostname must match a DNS-safe pattern.

A request that fails validation fails in seconds with a message that names the field, long before any clone exists to clean up. The field confirmed the design from the other direction too: a form request for the db-standard profile, with its 8 vCPU specification, was refused fast and clean by the `vm_max_vcpu` assert, exactly the behavior chapter 2 promised.

0

webhooks and schedules for the first delivery. Both exist in Semaphore, both can pre-fill survey values, and a customer self-service portal behind a webhook stayed a candidate until Phase 5's product track examined it against the runner the lab already trusts.

The token: rotation and receipts

The Proxmox API token is the pipeline's only real secret, and the campaign happened to exercise its whole lifecycle, because the token's value had crossed a chat window twice and deserved retirement. The rotation script, `scripts/pve-bootstrap.sh` from Phase 1, was advertised as idempotent, and the field found the one path that claim never covered: on PVE 9.x, `pveum user list` prints a bordered table, the script's awk-based exists-guard did not match it, and the script died at step one with "create user failed: user 'terraform@pve' already exists". Finding #15's fix makes both guards word-based, format-agnostic across plain, bordered, and JSON output, and the rotation recipe itself is now the explicit pair: `pveum user token remove`, then `pveum user token add`, no script re-run required.

The verification receipt taught three lessons in one afternoon, and the honest way to record them is to admit two of them were mine. A `pvesh` one-liner with the token in double quotes died in bash history expansion on the exclamation mark. The corrected single-quoted form reached `pvesh` and was refused: this PVE build's `pvesh` does not expose an `--api-token` flag. The next suggestion, a Bearer header, was simply the wrong scheme; Proxmox tokens authenticate with the `PVEAPIToken` scheme, and the working receipt, run by the operator from `pve-a`, looks like this:

THE TOKEN RECEIPT, AS FILED ON 29 AUGUST 2026 (ON PVE-A)

```
root@pve-a:~# curl -k -s \
-H "Authorization: PVEAPIToken terraform@pve!provider=" \
https://127.0.0.1:8006/api2/json/version
{"version":"9.2.2","release":"9.2"}
```

After a rotation, what must change is a shorter list than anxiety suggests. Mandatory, and first: the `PROXMOX_TOKEN_SECRET` value in the Semaphore Variable Group, because that is the live pipeline consumer through the #8 extra-var semantics; the token ID and user do not change. Optional, and only if Terraform will run again: the `tfvars` copies on `ctrl-01` and `pve-a`, a one-line `sed`. And the authoritative gate

is the one the pipeline already carries: the next run's token assert fails fast, in about two seconds, with a message that names the empty secret if the Variable Group still holds the old value. The receipt commands are a convenience; the guard is the guarantee.

One more object joined the project after the campaign, and it earns a pointer here because it reuses the same wiring: the Health Audit template (Phase 5b) runs 08_health_audit.yml against the same inventory and variable group, with no survey, twice a day on a schedule. Its story belongs to the Phase 5 records; the wiring lesson belongs to this chapter, because it proved the five objects above are a foundation, not a one-form trick.

6. First Deploy, Two Ways

The pipeline can prove itself twice: once on the CLI, where the failure modes are plain, and once through the form, which is the point of the phase. The CLI run is recommended first, not as ceremony but because a broken UI wiring and a broken playbook look identical in a task log, and the CLI removes the UI from the variables. The field campaign inverted the order anyway, form first, and paid for it with finding #8; the transcript ledger later in this chapter is the receipt that both orders converge to the same green.

The CLI smoke test

```
CTRL-01 (OR SEM-01): ONE CUSTOM VM, SMALL DISKS, NO UI INVOLVED

export PROXMOX_HOST=192.168.122.11 \
PROXMOX_USER=terraform@pve \
PROXMOX_TOKEN_ID=provider \
PROXMOX_TOKEN_SECRET=
cd ansible
ansible-playbook -i inventory/provisioning.yml \
playbooks/deploy_vm.yml \
-e vm_profile=custom -e vm_hostname=vm-faz4-01 \
-e vm_ip=192.168.122.60 \
-e vm_vcpu=2 -e vm_ram_gb=4 \
-e 'vm_disks=[{"role":"data","size_gb":10}]'
# the run ends with the 07 delivery receipt; the receiver-side proof:
# grep faz4-vm-faz4-01 /tmp/qradar-stub.log (on ctrl-01)
```

The playbook chain shows its shape in the recap: the localhost plays run first, the clone task reports the new VMID, the disk loop labels itself scsi1 data01 10G, and then the story crosses into the guest: guest agent, storage, Zabbix, rsyslog, the stub installers, and the delivery gate. A green run prints the receipt line that ends with the receiver-side instruction, and the grep on ctrl-01 is the field receipt: the faz4 tag, the hostname, the timestamp. The tag pattern continues the series convention from Phase 3; a delivery check message is distinguishable from noise in the stub log for exactly as long as someone needs to find

it. The exact shape a real green run prints, recap and all, is run #37's, printed later in this chapter.

Through the form

The second run is the demonstration. In Semaphore: New Task on the Deploy VM template, the survey appears, fill the seven fields, Run. The task page shows the run live; Semaphore streams playbook output as it happens, and the run ends on the same delivery receipt. The task record now holds the complete audit entry the manual process never had: who launched it, when, every form value, the full output, and the duration. That record is the answer to a question the voice memos asked without asking: when something ships wrong, the first question is not "who did what" but "which parameters did the machine actually receive", and the answer is one click deep.

The field's first form submission is worth quoting as a specimen of what a fresh wiring meets. The values that eventually worked were the lab's smallest honest shape: profile custom, hostname vm-faz4-01, IP 192.168.122.60, 2 vCPU, 4 GB RAM, one 10 GB data disk. The first attempt at db-standard, with its forced 8 vCPU / 32 GB, died at the node cap (finding #12, chapter 2); the first attempt at 2 GB RAM was refused by the enum set itself, because 2 is not in the allowed RAM ladder and the form declined to offer it. Both refusals were working as designed, which is the point: the pipeline's edges announced themselves before anything expensive happened.

Re-runs are safe by construction, and the campaign proved it the hard way, twice, before it proved it the intended way. The existence guard finds the VM, the clone is skipped, the VMID is adopted from the lookup, the disks are already attached, the VM is already running, and the guest plays converge with changed counts at zero. The one path that needed a field fix is the specification re-apply: sending net0 without a macaddr makes PVE regenerate the MAC on the running VM, and the controller's SSH wait dies for the full 600 seconds. That is finding #17, and its story, the probes, the mechanism, and the one-line pin that retired it, follow the run ledger below. With the pin in place, an adoption re-run of the whole chain completes in under a minute. One caution travels with the claim: re-running with a DIFFERENT profile against the same hostname re-applies CPU, memory, and tags to the existing VM, and disk changes flow through the same create: regular path; that is reconfiguration, and it is what the guard is for, but it is not a rename or a rebuild, and a shrink against a running VM should be a destroy and a fresh request (finding #13).

The proving runs (28-29 August 2026)

Edition 1 of this chapter ended at "run the form and read the receipt". The field wrote a longer chapter: twenty-plus runs across two days, thirteen numbered findings, and a green that arrived only at the last of them. The ledger below is the campaign in one table, run numbers as Semaphore's task history records them, and it is printed here for the same reason every transcript in this series is printed: the next person to wire a pipeline should see the failures the design survived, not just the architecture that resulted.

Run	When	What happened	What it taught
#1 - #6	28 Aug, evening	six failed runs: a YAML colon in a task name (#7), the empty token on the first form run (#8), a start timeout (#9), the ide2 PVE 500 (#10), the cloud-init ISO hang on NFS (#11), the node vCPU cap (#12), and a hot-applied spec wedge (#13)	the field tests the whole path, not the happy pieces; every fix became a guard, a timeout, or a comment in 00
#17	28 Aug, 23:57	play 00 green (ok=18 changed=5), the VM booted, SSH answered, play 01 reached the guest for the first time, and died at qemu-guest-agent apt: dpkg lock held by pid 3290	first proof of clone, cloud-init, and SSH; the first-boot apt race, finding #14
#30	29 Aug, 13:38	play 00 fully green on a fresh clone (VMID 103, web-small shape); the v2 name-based gate burned all 60 retries against the permanent unattended-upgr daemon, 13:38 to 13:43:58	finding #16: gate on locks, not process names; the v3 gate shipped the same day
#31	29 Aug, 17:06	plays 00-03 green on a true first boot in 35 s (SSH up in ~23 s; vm-faz4-01 ok=29 changed=7, data01 built to /srv/data); died at 04's pre-task assert	finding #18: Semaphore runs its own inventory, and the repo's group_vars never loads on the deploy path
#32	29 Aug, 17:28	first adoption run: clone skipped, VMID adopted, spec re-applied in 3 s, disks no-op; then Wait-for-SSH dead for the full 600 s	finding #17: the net0 re-send without a macaddr regenerates the MAC on the running VM
#33	29 Aug, 17:46	fresh clone (the previous VM had been destroyed outside the pipeline), old code checked out: SSH fine in 22 s, died at the same 04 assert	finding #18 is deterministic, not flaky; adoption vs fresh is the meaningful split, not old vs new VM
#36	29 Aug, 18:34	first patched run (a841f36): adoption with the MAC pin, Wait-for-SSH ok in ~10 s, 04's assert passed for the first time in the pipeline; died at apt_repository argument validation	findings #17 and #18 field-proven fixed; finding #19: lock_timeout is not an apt_repository parameter
#37	29 Aug, 18:54	FIRST FULL GREEN: localhost ok=16 changed=2 skipped=2, vm-faz4-01 ok=88 changed=11 failed=0; the delivery tag landed at the receiver at 15:54:43Z	the closing receipt, chapter 7; the campaign's every finding closed and re-run to green

Table 8. The proving-run ledger. Failed runs are not deleted from Semaphore; the red entries are the audit trail of exactly what the pipeline refused, and when.

Two habits made this ledger readable at all, and both are worth copying. First, no destroy: a failed run leaves a half-configured VM that is also a test fixture, and the adoption path turns the NEXT run into a convergence check instead of a repeat of the same first boot. The runs that ignored this rule, deleting the VM between attempts, cost themselves the strongest test case the campaign had. Second, the red entries

stay: a refusal that dies before the clone with changed=0 and a clean qm list is not litter, it is the audit record of the gate doing its job.

Finding #18: the inventory Semaphore never loads

Run #31 died at the first task of play 04, and the shape of the death was the interesting part: the same playbooks had run green from the CLI all through Phase 3, with the same variables file, on the same machines. The assert said the zabbix variables were undefined; the file defining them sat in inventory/group_vars/provisioned.yml, exactly where the guide had told the reader it sat. The mechanism, once read out of the run log, is a deploy-mode fact this series had never crossed before: Semaphore does not run ansible with the repository's inventory file. It installs its own generated inventory, with a warning about empty hosts, and that inventory has no provisioned group for group_vars to attach to. The add_host hostvars DO arrive; that is why plays 01 through 03 passed, data_disks rode through on the hostvars. Group-scope variables do not. Static CLI runs and deploy runs had been reading two different variable universes all along, and 04's assert was the first instrument on the path that noticed.

RUN #31, PLAY 04 PRE-TASK, 29 AUGUST 2026 (KEY LINES)

```
TASK [Require the agent layer settings] *****
fatal: [vm-faz4-01]: FAILED! => {
  "assertion": "zabbix_agent_version is defined",
  "evaluated_to": false,
  ...
}

# the file defining zabbix_agent_version sat, at that moment,
# in inventory/group_vars/provisioned.yml: loaded on every
# static CLI run, never loaded on the Semaphore deploy path.
```

The fix moved the agent layer, zabbix, rsyslog, and the security stubs, into ansible/playbooks/vars/agent-layer.yml, loaded by plays 04, 05, 06 and 07 through vars_files, the same pattern play 00 had always used for profiles.yml. One file, both run modes: static runs load it because the plays ask for it, deploy runs load it because the plays ask for it. Storage stayed in group_vars on purpose, because data_disks arrives as an add_host hostvar on the deploy path and a vars_files copy would shadow it. The group's empty declaration in provisioning.yml still matters for the static path, so it keeps its comment and its row in the troubleshooting matrix, now with the honest explanation. The patch was seven files, and it was proven by roundtrip before it shipped: the patcher, reversed onto a scratch copy, reproduced the pre-fix tree byte for byte, then applied again reproduced the fixed tree byte for byte.

Finding #17: the adoption path that stopped answering

Run #32 was the adoption path's first field exercise, and for four tasks it was a textbook demonstration: lookup found the VM, the clone skipped itself, the ide2 guard held, the disk attach returned its first-ever clean ok instead of changed, the spec re-apply finished in three seconds. Then the run parked at Wait for

SSH, port 22 against an address that had answered SSH in 23 seconds the hour before, and stayed parked for the full 600 seconds until the task failed on timeout. The reproductions that followed drew the line cleanly: fresh clone, SSH every time; adoption, dead every time.

THE SAME TASK, BEFORE AND AFTER THE PIN (KEY LINES)

```
# run #32, 17:28, adoption path, code before the fix:
TASK [Wait for SSH (cloud-init brings the interface up on
first boot)] *****
... 17:28:34 -> 17:38:34, every poll unanswered ...
fatal: [localhost]: FAILED! => {"changed": false,
"elapsed": 600, ...}

# run #36, 18:35, same VM, same path, net0 now carries
# macaddr= on the adoption re-apply:
TASK [Wait for SSH (cloud-init brings the interface up on
first boot)] *****
ok: [localhost] => {"changed": false, "elapsed": 10}
```

The probes narrowed the mechanism more honestly than the first theory did. The VM stayed pingable from the node; the guest agent answered network-get-interfaces; sshd was listening on 0.0.0.0:22; the guest's eth0 MAC, the config's net0 MAC, and the node's ARP entry all agreed. So "the guest loses its IP", the first edition's working theory, was disproven by the field: the guest keeps its address, and what breaks is the controller-to-guest SSH path, transiently, in the window after a net0 update that carries no macaddr. The precise internal mechanism went uninstrumented, evidence destroyed by the out-of-band VM deletion between runs, and the guide records that honestly rather than inventing a cleaner story. The fix stands on narrower, sufficient ground: pin the existing MAC, extracted from the lookup's net0 with a non-capturing regex, and the disruption source disappears; 600 seconds became 10, and every adoption run since has answered. The regex detail earns its sentence because the capturing version of the same pattern returns the group, "A1:", instead of the MAC, and that bug was caught by the patch's own validation script before the field ever saw it.

Run #37: the first full green

The #19 fix, two deleted lines, went in on the evening of 29 August, pushed with the same rev-parse-equals-ls-remote receipt as every patch before it, and the next run needed no ceremony: the same form values, the same VM, the adoption path. Plays 00 through 03 converged with ok-heavy, changed-light recaps; plays 04, 05 and 06, each on its vars_files-loaded settings, ran for the first time inside the pipeline and changed exactly what a first arrival changes; and play 07 read the whole machine back one gate at a time until the delivery receipt printed.


```
RUN #37, THE PLAY RECAP, 29 AUGUST 2026, 18:54 LOCAL (AS FILED)
```

```
PLAY RECAP *****
localhost : ok=16 changed=2 unreachable=0 failed=0
skipped=2 rescued=0 ignored=0
vm-faz4-01 : ok=88 changed=11 unreachable=0 failed=0
skipped=0 rescued=0 ignored=0
```

Every number in that recap has a reading. Zero failed and zero unreachable on both hosts is the headline: the first run of the entire campaign in which no play died. The guest's skipped=0 says every gate in plays 01 through 07 executed; nothing took an early exit. localhost's skipped=2 is the adoption path itself, the clone and the fresh-VMID record standing down because the VM already existed, and its changed=2 is the re-adoption bookkeeping. The guest's changed=11 is the arithmetic the campaign had been chasing: five changes from play 04 (the repo, the key, the package, the config, the restart), three from play 05 (the forward rule, the restart, the smoke-test file), and three from play 06 (the directories and scripts, the installer runs, the marker writes). The same 5+3+3 the standalone Phase 3 runs printed a day earlier, from a different executor on a different machine: the playbooks are portable, and the arithmetic travels with them.

One cross-check closes the run's file. The recap line printed at 18:54:44 local time, Europe/Istanbul, UTC+3: 15:54:44Z. The receiver's log, grep and all, holds the delivery tag at 15:54:43Z and the pipeline's own grep-verify command at 15:54:44Z. The recap and the receipt belong to the same second of the same run; there is no gap in which anything could have been different. That is what "field-proven end to end" means when it is written in the roadmap, and chapter 7 shows the receiver's side of that second.

7. The Delivery Gate: 07_delivery_check.yml

Every phase in this series ends by proving what it built. Phase 4 automates that instinct into the pipeline itself: the last play of every deploy is a checklist whose green output is the handover record. A VM that fails the gate is not delivered, even if every earlier play was green; that is the entire point of putting the gate last.

Check	How it is proven	Why this and not something weaker
Services active	service_facts plus asserts on zabbix-agent2, rsyslog, qemu-guest-agent	state: running in a play is a request; service_facts is an observation
Mounts up	ansible_facts.mounts must contain every data disk's mount point	a built filesystem that is not mounted is a DB server waiting to fail
Agent markers	every security_agents entry's .installed file must exist	06's marker contract; the marker is the truth, not the installer output
Forwarding established	ss -tn state established against the receiver port, with retries	the disk queue buffers when the link is down; delivery means the link is up
Tagged message	logger with tag faz4-, found in the local log	proves rsyslog actually processed a message, not just that it is running
Zabbix listening	ss -tln on port 10050	active checks need the listener; server-side registration is a Zabbix-side action

Table 9. The six checks of the delivery gate.

The gate reads its expectations from the same agent-layer file the plays above it wrote against, vars/agent-layer.yml through vars_files, so its expectations cannot drift from its build in either run mode; the #18 story in chapter 6 is why that sentence had to change in this edition. A failure stops the pipeline loudly, with a fail_msg that names the diagnostic command for the failing check; the series' position on quiet warnings is unchanged since Phase 2. What the gate deliberately does not check is anything outside the VM: the Zabbix server's knowledge of the host, the QRadar-side ingestion, the licensed agents' real health. Those are other systems' jobs, and pretending to verify them from the guest would be the dishonest kind of green.

The receipt at the receiver (29 August)

Two of the gate's checks point outside the VM, at the receiver, and the campaign's one precondition was that the receiver actually be listening when the pipeline first needed it. The lab's stand-in is the Phase 3 socat stub on ctrl-01, one process, one port, one file. The ten-second check, run inside ctrl-01, is the receipt that the precondition holds:

THE RECEIVER PRECONDITION, 29 AUGUST 2026 (ON CTRL-01)

```
root@ctrl-01:~# ss -tlnp | grep 5514
LISTEN 0 0 0.0.0.0:5514 0.0.0.0:* \
users:(("socat",pid=1822,fd=5))

# socat pid 1822, bound to every interface, writing to
# /tmp/qradar-stub.log; an optional end-to-end selftest:
# echo selftest | socat - TCP:192.168.122.10:5514
# then tail /tmp/qradar-stub.log
```

With the receiver live, the closing receipt is one grep. The transcript below is the campaign's last filed artifact, trimmed to the lines that matter: the delivery tag itself, the pipeline's own commands that wrote and verified it (every ansible command invocation is itself forwarded by the *.* rule, so the pipeline signs its own paper), and, after it, the ordinary traffic that proves the forward is a living connection and not a one-shot.

THE CLOSING RECEIPT, 29 AUGUST 2026 (ON CTRL-01; TRIMMED)

```
root@ctrl-01:~# grep faz4-vm-faz4-01 /tmp/qradar-stub.log
<13>Aug 29 15:54:39 vm-faz4-01 ansible-command: Invoked \
... logger -t faz4-vm-faz4-01 "delivery check ..."
<13>Aug 29 15:54:43 vm-faz4-01 faz4-vm-faz4-01: \
delivery check 2026-08-29T15:54:39Z
<13>Aug 29 15:54:44 vm-faz4-01 ansible-command: Invoked \
... grep -F faz4-vm-faz4-01 /var/log/syslog
<13>Aug 29 15:59:.. vm-faz4-01 PackageKit: ...
<13>Aug 29 16:00:.. vm-faz4-01 sysstat: ...
```

Read the receipt the way an auditor would. The tag arrives four seconds after the logger command that wrote it, and the local grep that verified it inside the VM arrives one second after the tag, and the recap printed in the same second, 15:54:44Z, back in chapter 6. Because play 07 runs its checks in order, services, mounts, markers, the ESTABLISHED socket, then the logger, then the local grep, and only then the delivery message, the tag's arrival at the receiver is proof that every earlier gate in the play passed. And the tail of the file is the quiet proof that keeps proving: PackageKit at 15:59, sysstat at 16:00, the machine's ordinary voice, still arriving. A forward configured once, delivering continuously, is what the QRadar pattern was always for.

The receipt also closed the campaign's books on a question of scope: the receiver is a stub, and the stub does not parse, alert, or store beyond one file. Nothing in the pipeline pretends otherwise. What the receipt proves is the contract the pipeline owns: the guest forwards everything, over TCP, with a disk queue behind it, and the bytes arrive. What the receiving system does with them is the receiving system's job, in this lab and in production.

8. Troubleshooting

The matrix follows the series convention: the error as the field will see it, the cause, and the fix. Edition 1 carried fifteen rows and called six of them field-found, which felt like a lot at the time. The campaign added thirteen more, findings #7 through #19 plus the token receipt saga, and every one of them is printed below with the fix that shipped. The rows are grouped in the order the series met them: eight from the sandbox, the first install, and the guide's own chapter 4 (28 August); six module-behavior and logic-test rows; then the campaign block. The lesson the counts teach is the one the Phase 3 guide already wrote: a sandbox that exercises the happy path tests the binary, and the field tests the boundaries, the

systemd unit, the inherited environment, the login endpoint, the generated inventory, and the printed page. Test the whole path, or the path will test you.

Symptom	Cause	Fix
git push answers "error: src refspec main does not match any" at the chapter 4 step (field-found on ctrl-01, 28 Aug 2026; guide errata)	plain git init still creates branch master (the default changes only in Git 3.0) while the bare repo, the push command, and the Semaphore repository object all say main	on the already-committed copy: git branch -m master main, then push. The corrected chapter 4 block uses git init -b main from the start
authorized_keys ends up empty; a later clone from sem-01 answers "Permission denied (publickey)"; copy-pasting the original block also produced "chmod: missing operand" (field-found on ctrl-01, 28 Aug 2026; guide errata)	two errata in the original chapter 4 block: the placeholder was silently eaten by the PDF renderer (esc() escaped nothing, and the parser treats as markup), and the 89-character chown+chmod line wrapped across two PDF lines, so the pasted chmod ran without its operand	this edition writes the lab's real deploy key into the block, keeps every line short enough not to wrap, and esc() escapes for real; ad-hoc: echo the key to the file, then chown and chmod as separate lines
sem-01-setup.sh stops before step 1/8, after "tr: write error: Broken pipe"	head -c 16 closes the pipe after 16 bytes; tr dies on the write error; under set -euo pipefail the status rides the command substitution into the password assignment and aborts the script (field-found on sem-01, 28 Aug 2026)	fixed in the script: a bounded 256-byte read, then cut. Workaround on an already-pushed copy: pass the password as the script's first argument
ansible-galaxy inside sem-01 dies with "Ansible could not initialize the preferred locale: unsupported locale setting"	pct exec inherits the calling shell's LANG (en_US.UTF-8 on pve-a), which the minimal container has not generated; ansible-core refuses an invalid locale. The perl warnings in the same output are the cosmetic cousin (field-found on sem-01, 28 Aug 2026)	fixed in the script: LANG/LC_ALL exported to C.UTF-8, the locales package installed, en_US.UTF-8 generated. Ad-hoc: env LANG=C.UTF-8 in front of the command

Symptom	Cause	Fix
journalctl shows "Cannot Find configuration! Use --config parameter to point to a JSON or YAML file"; the unit loops on activating (auto-restart), restart counter 445 after one hour	the message is the binary's catch-all for an UNREADABLE config, not only a missing one: root-created state meets a non-root unit. The migrations and the admin creation ran as root, so the database is root-owned and config.json is 640 root:root; the User=semaphore unit can read neither (field-found on sem-01, 28 Aug 2026)	fixed in the script: the config is made group-readable by the service user and the data directory is handed over recursively after the migrations. Ad-hoc: chown root:semaphore config.json; chown -R semaphore:semaphore /var/lib/semaphore; restart
First sign-in shows "Failed to create session"; POST /api/auth/login answers HTTP 500 although the password is correct (field-found on sem-01, 28 Aug 2026)	v2.19 signs session cookies and requires two secrets in config.json: cookie_hash and cookie_encryption. The older field names, cookie_hash_key and cookie_encryption_key, still circulating in guides, are silently ignored; semaphore setup generates the keys, a hand-written minimal config does not	fixed in the script: both keys generated with pipefail-safe od -N and written into the config. Ad-hoc: add the two keys, restart. Verified round trip: login 204 with Set-Cookie, authenticated GET /api/user 200
json: cannot unmarshal number into Go struct field ConfigType.port of type string	config.json has "port": 3000 (a number); the binary requires a string	quote it: "port": "3000". sem-01-setup.sh writes it correctly
Semaphore .deb installed but no service exists	the .deb ships only /usr/bin/semaphore, no unit, no config	the setup script creates /etc/semaphore/config.json and the systemd unit from the official template
Clone succeeded but the VM has the template's CPU, RAM, and network	proxmox_kvm's clone branch forwards only format, full, pool, snapname, storage, target; spec parameters are dropped	the two-task pattern: clone first, then update: true (with update_unsafe: true for net and ide); never rely on specs riding the clone call
Re-ran the deploy; a second VM with the same name appeared	clone mode allocates the next free VMID without checking names	the playbook's proxmox_vm_info guard prevents this; if you bypass it, remove the duplicate VM and re-run
pip install fails with externally-managed-environment on sem-01	PEP 668 on Ubuntu 26.04; the Phase 1 finding, same wall	the setup script uses apt packages (ansible, python3-proxmoxer, python3-requests); a venv under the service user is the fallback

Symptom	Cause	Fix
Syntax error in template: expected token ',', got 'for'	a Jinja dict literal inside a list comprehension breaks the parser	the derivation uses a loop with list append, the same pattern 02_storage.yml always used
'final_data_disks' is undefined on a diskless profile	the derivation loop skipped itself on an empty disk list, leaving the fact unset	the playbook initializes the list to empty before the loop; caught by the web-small logic test
Serial not visible in the guest after attaching a disk to a RUNNING VM	the Phase 2 field finding: a new disk's serial may reach the guest only at the next full VM start	qm shutdown and qm start on the PVE node; a guest reboot is not enough. The Phase 4 path attaches disks before first start, so first deploys never hit this
Deploy run dies in 04's pre-task assert: zabbix_agent_version is undefined, although static CLI runs of the same plays pass (field-found, run #31, 29 Aug 2026)	Semaphore runs ansible with its OWN generated inventory, so the repo's inventory/group_vars/provisioned.yml never loads on the deploy path; add_host hostvars (data_disks) DO arrive, which is why plays 01-03 passed, but group-scope agent vars do not	the agent layer lives in ansible/playbooks/vars/agent-layer.yml, loaded by plays 04/05/06/07 via vars_files; group_vars keeps storage only; one source of truth for both run modes. The empty provisioned group in provisioning.yml still matters for static runs, keep it
First form-driven run dies in seconds at the token assert: PROXMOX_TOKEN_SECRET is empty although the Variable Group plainly holds it (field-found, 28 Aug 2026)	finding #8: Semaphore v2.19 delivers Variable Group JSON values to the playbook as EXTRA-VARS, not as process environment variables, so lookup(env) alone finds nothing	vars/lab-environment.yml reads each secret as env var or extra-var of the same name; either delivery path feeds the pipeline
YAML parse error loading 00_provision_vm.yml, pointing at a task name line (field-found, 28 Aug 2026)	finding #7: an unquoted colon inside a task name makes the YAML parser read the name as a mapping	quote the name, or drop the colon; the shipped file quotes every task name that carries one
The start task dies at ~30 s on a fresh clone while the console shows "generating cloud-init ISO" (field-found, 28 Aug 2026)	finding #9: the proxmox_kvm start timeout defaults to 30 s, too tight while ide2's ISO is still being written	timeout: 300 on the start task
Re-run answers HTTP 500 from PVE and the log mentions ide2 already existing (field-found, 28 Aug 2026)	finding #10: re-applying ide2 re-ALLOCATES the cloudinit disk; the create form is not the update form	the vm_has_cloudinit guard (config: current in the lookup) sends ide2 only when the VM does not carry one yet

Symptom	Cause	Fix
qm start hangs for minutes at "generating cloud-init ISO" when the drive lives on nfs-shared, reproducibly (field-found, 28 Aug 2026)	finding #11: ISO regeneration on NFS holds a lock qm start does not wait for kindly; the hang is deterministic, not load	proxmox_cloudinit_storage: local-lvm in vars/lab-environment.yml; the ISO is small and lock-sensitive, local storage is its right home
qm set accepted the spec but qm start refuses it: the node policy answers with a per-VM vCPU cap (field-found, 28 Aug 2026)	finding #12: the node enforces its cap at START time only, so the spec looks fine until the boot	vm_max_vcpu in lab-environment.yml, asserted in 00 before the clone, so the refusal costs seconds and names the cap
A hot-applied spec change (a memory shrink) against a running VM wedges the VM (field-found, 28 Aug 2026)	finding #13: the update path re-applies values, and a running VM is not a reconfigure-everything target	for real spec changes, destroy and re-provision; re-runs re-apply identical values only, by design
Run dies in play 01 at the first apt task with rc=100 and dpkg lock errors on a fresh VM; or the gate burns every retry against unattended-upgr (field-found, runs #17 and #30, 28-29 Aug 2026)	findings #14 and #16: first-boot cloud-init and unattended-upgrades hold the dpkg lock in waves, and a process-NAME gate can never clear because unattended-upgr is a permanent daemon on Ubuntu 26.04	gate v3 (tasks/wait_for_apt.yml): cloud-init status --wait, then fuser on the lock files and systemctl is-active on the apt-daily units; plus lock_timeout: 1800 on the apt call sites
Token rotation dies at step 1: "create user failed: user 'terraform@pve' already exists" (field-found, 29 Aug 2026)	finding #15: pveum user list prints a bordered table on PVE 9.x, and the awk exists-guard did not match it, so the add ran and set -e killed the script	both guards are now word-matched (grep -w), format-agnostic; rotation itself is the explicit token remove + add pair, no script re-run
Adoption re-run: the VM stays pingable, the guest agent answers, sshd listens, but Wait for SSH dies at the full 600 s (field-found, runs #32/#33, 29 Aug 2026)	finding #17: the spec re-apply resubmits net0 without a macaddr, PVE regenerates the MAC on the running VM, and the controller-to-guest SSH path breaks transiently; the guest keeps its IP	the lookup extracts the existing MAC (non-capturing regex) and the apply pins it with macaddr= on the adoption path; 600 s became ~10 s
Run #36 shape: 04's assert passes, then "Add the pinned Zabbix apt repository" dies in module argument validation: Unsupported parameters for (apt_repository) module: lock_timeout (field-found, 29 Aug 2026)	finding #19: the #14v2 patch added lock_timeout to all five apt call sites, but apt_repository does not accept it (only apt does); latent until play 04 first reached the task inside the pipeline	the two lines were removed from the apt_repository task only; the four real apt call sites keep lock_timeout, and the v3 gate runs immediately before in the same play

Symptom	Cause	Fix
Token verification one-liners keep failing: bash answers "event not found", pvsh answers "Unknown option: api-token", and a Bearer header is refused	receipt-command traps, two of them mine: the exclamation mark in the token ID triggers bash history expansion in double quotes; this pvsh build has no --api-token flag; and PVE tokens do not use the Bearer scheme	the working receipt: curl -k with "Authorization: PVEAPIToken terraform@pve!provider=" against /api2/json/version, expecting the version JSON; or skip the receipt and let the next run's token assert be the gate

Table 10. The Phase 4 troubleshooting matrix: twenty-seven rows, every field finding kept with its fix.

9. Where the Project Stands

Edition 1 of this chapter was a wishlist titled "What Phase 5 Holds", written the day Phase 4 shipped its plan. The field has since answered it, in full, and the honest edition 2 replaces the wishlist with the ledger. Phase 4 closed on 29 August 2026 with run #37's recap and the receiver receipt printed in chapters 6 and 7. Phase 5, scoped within days as three legs, closed within two more days, each leg field-proven the same way this guide documents everything: with runs, refusals, and receipts.

- Phase 5a, admission control: the gate the form never had. Play 00 now checks IP collisions against every VM's ipconfig0, probes the address for a squatter, and sums the fleet's allocated vCPU and RAM before the clone. All four branches proven in the field on 30 August: a grant with live budget math, a collision refusal naming the owning VMID, a squatter refusal at an address that answered TCP/22, and a budget refusal at 7 of 8 vCPU and 23 of 16 GB. Every refusal died before the clone with zero qm footprint; the red Semaphore entries are the audit trail, kept on purpose.
- Phase 5b, the health audit: 08_health_audit.yml runs read-only checks across the cluster and the fleet, writes a baseline, diffs against it, prints a digest to the task log always, and mails it when msmtpt is configured. The chain was proven end to end: baseline written, drift-zero proven, the digest landed in the inbox, a scheduler-driven fire produced the quiet-fleet 0/0/0 digest at Turkish 07:00 and 19:00, and when the mail credential was later revoked the mail step failed loudly by design, then retired gracefully to log-only.
- Phase 5c, concurrency: the question was whether two simultaneous forms could race the admission gate. The field answered that they cannot even try: Semaphore's task queue runs one task at a time, so the queue is an accidental admission mutex. A Health Audit fired mid-deploy likewise waited its turn. The check-then-act race stays reachable only via CLI or API, and the documented defense there is the post-apply re-verify, defense in depth rather than lab requirement. A deleted-VM detector and its self-heal were proven in the same window, and a custom-UI track was examined and dropped: Semaphore stays the only face.

The backlog that survives is shorter and more honest than the original wishlist. TLS on the log path (port 6514) remains the next real work on the forwarding contract. The resize button, a template wrapping

proxmox_disk state: resized plus a re-run of 02_storage.yml, remains a one-evening build on the field-proven growth chain. Load testing, the honest EPS limit of the forward path, remains unmeasured. HA and remote runners remain a documented env-var switch away, deliberately unwalked. The real agents, licensed binaries behind the marker contract, remain a swap-in the Phase 3 guide already scripts. And the AWX watch stays open: if a 25.x release revives the project, the engine is UI-agnostic by design and the migration is a wiring exercise, not a rewrite.

What the series has standing, as this guide goes to press, is two observers and a rule. The receiver stub on ctrl-01 collects every managed VM's logs, continuous and timestamped, and has caught everything from delivery tags to out-of-band reboots. The health audit runs twice a day against its own baseline and remembers what the fleet owed it, including the machines that vanished. The rule is the one every phase of this series kept when it would have been easier to break: claims get verified, errors get kept, and the receipts get filed. The whole story, five phases of it, is told once more in the series retrospective; the guides you are holding are the receipts.